TP 1 – Installer et lancer Apache Airflow

Cours Apache Airflow — support stagiaire

> **Prérequis** : le TP 0 doit être terminé (Docker Desktop, uv et VS Code installés et vérifiés).

Durée : 30 min si les images sont déjà téléchargées, jusqu'à 1 h sinon **Objectif** : faire tourner Airflow avec Docker et préparer un environnement Python pour écrire des DAGs avec l'autocomplétion.

Pourquoi Docker ?

Airflow n'est pas **un** programme mais **plusieurs composants** qui doivent tourner ensemble :

- une **base de données** (Postgres) qui stocke l'état des pipelines,
- un **scheduler** qui décide quoi lancer et quand,
- un **DAG processor** qui lit les fichiers de pipelines,
- un **API server** qui fournit l'interface web,
- un **triggerer** pour les tâches en attente d'un événement.

Docker les livre prêts à l'emploi, reliés entre eux, identiques sur tous les postes, **comme en production**. Il permet aussi de faire tourner Airflow sous Windows, qu'il ne supporte pas nativement.

	Rôle
Docker	Fait tourner Airflow
venv + uv	Donne l'autocomplétion dans VS Code pour écrire les DAGs

Étape 1 – Préparer le dossier du projet

```
mkdir airflow-intro
cd airflow-intro
code .
```

Si `code` n'est pas reconnu : ouvrir VS Code → **Fichier → Ouvrir le dossier** → `airflow-intro`.

Copier dans ce dossier le fichier `docker-compose.yaml` fourni (glisser-déposer depuis Téléchargements vers la barre latérale de VS Code, option **Copier**), ou dans le terminal :

```
Copy-Item "$HOME\Downloads\docker-compose.yaml" .
```

> Vérifier le nom exact : `docker-compose.yaml` ou `docker-compose.yml`, sans `(1)` ajouté par le navigateur (renommer avec `F2`).

Créer les sous-dossiers utilisés par Airflow :

```
mkdir dags, logs, plugins, config
```

Sous Windows, créer aussi le fichier `.env` (supprime un warning sans conséquence) :

```
Set-Content .env "AIRFLOW_UID=50000"
```

Sous Linux / macOS : `echo "AIRFLOW_UID=$(id -u)" > .env`

Étape 2 – Repérer la version d'Airflow

Ouvrir `docker-compose.yaml` et chercher la ligne `image:` :

```
image: ${AIRFLOW_IMAGE_NAME:-apache/airflow:3.0.0}
```

Noter ce numéro de version (ici `3.0.0`). On installera exactement la même dans l'environnement Python.

Le fichier fourni utilise le **LocalExecutor** : les tâches s'exécutent dans le scheduler, sans worker ni Redis. C'est plus léger en

mémoire, idéal pour se former. Le fichier officiel du site d'Airflow contient en plus `airflow-worker`, `redis` et `flower` (CeleryExecutor).

Étape 3 – Créer l'environnement virtuel Python

Ouvrir le terminal intégré (`Ctrl + ù` ou **Terminal → Nouveau terminal**) et vérifier qu'on est dans `airflow-intro`.

```
uv venv --python 3.12
```

> Choisir **Python 3.12** (ou 3.11). Python 3.8 n'est plus maintenu et n'est pas supporté par Airflow 3. Attention à l'écriture : `3.12`, pas `3.87`. Si la version n'est pas sur la machine, uv la télécharge automatiquement.

Un dossier `.venv` apparaît dans la barre latérale.

Étape 4 – Activer l'environnement

Windows :

```
.\.venv\Scripts\activate
```

macOS / Linux :

```
source .venv/bin/activate
```

Le préfixe `(.venv)` apparaît en début de ligne.

> Erreur *« l'exécution de scripts est désactivée sur ce système »* → voir TP 0 étape 6.

Étape 5 – Installer Airflow dans l'environnement

> Un venv créé par uv **ne contient pas** `pip` → on utilise `uv pip install`. Airflow s'installe avec son **fichier de contraintes** officiel pour éviter les conflits de dépendances.

Adapter la version d'Airflow (étape 2) et celle de Python :

```
uv pip install "apache-airflow==3.0.0" --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-3.0.0/
```

Vérifier :

```
uv pip show apache-airflow
```

Relier VS Code à l'environnement : `Ctrl + Shift + P` → *Python: Select Interpreter* → `.venv`.

> Cette installation sert **uniquement à l'autocomplétion**. C'est Docker qui fait tourner Airflow.

Étape 6 – Vérifier qu'aucun conteneur n'entre en conflit

Docker Desktop → **Containers** : arrêter tout conteneur Airflow ou Postgres déjà en cours. Ports à surveiller : **8080** (interface Airflow) et **5432** (Postgres).

Étape 7 – Lancer Airflow

```
docker compose up -d
```

Le `-d` lance Airflow **en arrière-plan** : le terminal reste disponible. `airflow-init` (création de la base et du compte admin) s'exécute automatiquement avant les autres services.

Suivre le démarrage, en relançant la commande régulièrement :

```
docker compose ps
```

Statut	Signification
`health: starting`	Ça démarre, patience
`healthy`	Prêt
`restarting` ou `exited`	Problème → consulter les logs

Résultat attendu (fichier fourni) : 5 services `healthy` — `airflow-apiserver`, `airflow-dag-processor`, `airflow-scheduler`, `airflow-triggerer`, `postgres`.

> **Premier lancement** : 5 à 20 min selon la connexion (téléchargement de plus de 1 Go d'images), puis 2 à 4 min pour que tout passe `healthy`. **Lancements suivants** : moins d'une minute, plus 1 à 2 min de vérifications.

En cas de souci, afficher les logs en direct (`Ctrl + C` pour en sortir, Airflow continue de tourner) :

```
docker compose logs -f
```

Étape 8 – Se connecter à l'interface

Ouvrir **http://localhost:8080** (ou cliquer sur le port `8080` dans Docker Desktop).

- Identifiant : `airflow`
- Mot de passe : `airflow`

La page d'accueil d'Airflow s'affiche : installation réussie.

Étape 9 – Sortir proprement

Dans un terminal ouvert dans `airflow-intro` :

1. Arrêter Airflow :

   ```
   docker compose down
   ```

   > Ne pas ajouter `-v` : cela efface aussi la base de données.

2. Désactiver l'environnement Python (si `(.venv)` est affiché) :

   ```
   deactivate
   ```

3. Fermer VS Code.
4. Quitter Docker Desktop : clic droit sur l'icône près de l'horloge → **Quit Docker Desktop**. Sous Windows, c'est ce qui libère vraiment la mémoire (processus `vmmem`).

Ce qui est conservé après `docker compose down`

Élément	Conservé ?
Images Docker (le gros téléchargement)	
Base de données (historique, compte `airflow`)	(volume Docker)
Dossiers `dags/`, `logs/`, `plugins/`, `config/`	
Environnement `.venv`	
Conteneurs	Non recréés en quelques secondes

Ce qui provoquerait un retéléchargement : changement de version d'image, `docker system prune -a`, *Clean / Purge data*. Ce qui effacerait la base : `docker compose down -v`.

Relancer la fois suivante

1. Lancer Docker Desktop et attendre qu'il soit prêt
2. Dans `airflow-intro` : `docker compose up -d`
3. `docker compose ps` jusqu'à `healthy`, puis http://localhost:8080

Mémo des commandes

Commande	Effet
`docker compose up -d`	Démarrer Airflow en arrière-plan
`docker compose ps`	État des services
`docker compose logs -f`	Logs en direct (`Ctrl + C` pour sortir)
`docker compose down`	Arrêter en gardant les données
`docker compose down -v`	Arrêter et tout effacer (repartir de zéro)
`docker compose pull`	Télécharger les images à l'avance

Travailler avec les terminaux de VS Code

- **Ouvrir / masquer le panneau** : `Ctrl + ù` (masquer ne ferme pas les terminaux)
- **Nouveau terminal** : bouton `+` du panneau
- **Fermer un terminal** : cliquer dedans et taper `exit`
- **Tout fermer** : `Ctrl + Shift + P` → `kill all` → *Terminal : Tuer tous les terminaux*
- L'icône corbeille est en haut à droite du panneau, parfois cachée derrière `…` si le panneau est étroit

> Si Airflow a été lancé **sans** `-d`, le terminal affiche les logs en continu et n'accepte plus de commande. Cliquer dedans, `Ctrl + C` (deux fois si besoin), puis `docker compose up -d`. Si rien ne réagit : `docker compose down` depuis un autre terminal.

Dépannage TP 1

Symptôme	Cause probable	Solution
`No interpreter found for Python 3.87`	Version mal écrite	`uv venv --python 3.12`
`l'exécution de scripts est désactivée`	Politique PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`pip` introuvable ou installe au mauvais endroit	Le venv uv n'a pas de pip	`uv pip install ...`
`code` non reconnu	VS Code du Store	Fichier → Ouvrir le dossier, ou réinstaller via winget
`no configuration file provided`	Mauvais dossier ou fichier renommé	Se placer dans `airflow-intro`, vérifier le nom du fichier
`port is already allocated`	8080 ou 5432 déjà utilisé	Arrêter l'autre conteneur ou application
`The "AIRFLOW_UID" variable is not set`	Pas de fichier `.env`	Sans effet sous Windows ; créer `.env` (étape 1)
`Not time to refresh bundle dags-folder` en boucle	Message d'information du DAG processor	Normal, ignorer
Démarrage très long	Téléchargement des images au premier lancement	Patienter ; `docker compose pull` à l'avance
Terminal bloqué, impossible d'écrire	Lancé sans `-d`	`Ctrl + C`, puis `docker compose up -d`
`airflow-init exited with code 0`	Initialisation terminée	Normal
Conteneurs en `restarting`	Pas assez de mémoire	TP 0 étape 2 : 4 à 8 Go

Checklist TP 1

- ☐ Dossier `airflow-intro` avec `docker-compose.yaml`, `.env`, `dags/`, `logs/`, `plugins/`, `config/`
- ☐ `.venv` en Python 3.11 ou 3.12, sélectionné dans VS Code
- ☐ `apache-airflow` installé, même version que l'image Docker
- ☐ Services `healthy` dans `docker compose ps`
- ☐ Connexion réussie sur http://localhost:8080

- ☐ Arrêt propre avec `docker compose down`